

DrawMeet Architecture Notes

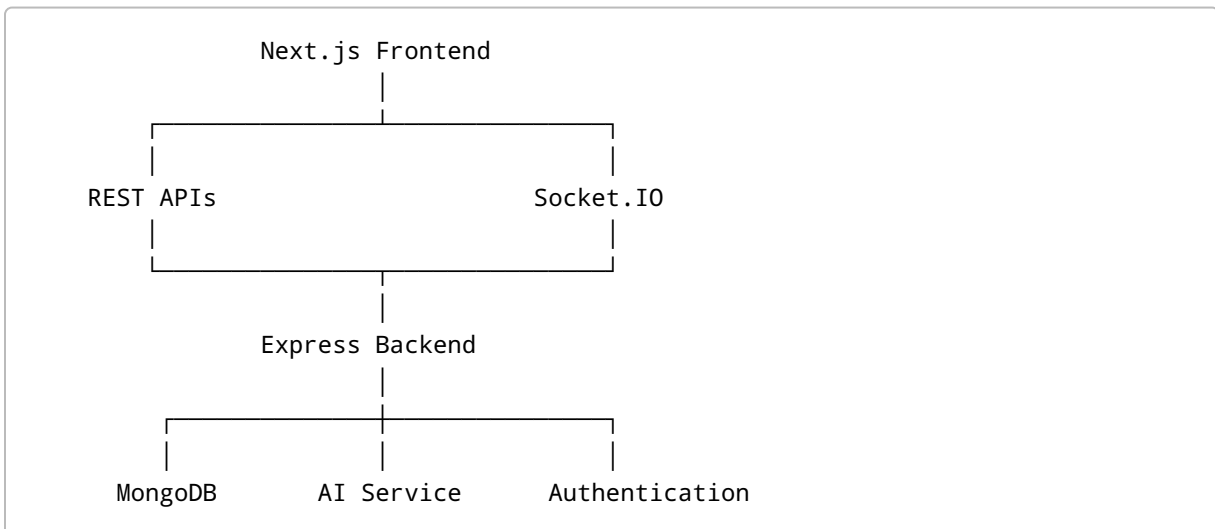
1. Project Overview

DrawMeet is a **real-time collaborative whiteboard** where multiple users can draw, chat, collaborate, invite members, and use AI features such as diagram generation and board summarization.

Tech Stack

- **Frontend:** Next.js, React, TypeScript, React Konva, TailwindCSS
 - **Backend:** Node.js, Express.js, Socket.IO
 - **Database:** MongoDB
 - **Authentication:** JWT
 - **AI Service:** Python FastAPI + LangChain + Google Gemini + Pydantic
-

2. High-Level Architecture



REST APIs are used for authentication, board management, AI features, replay, invitations, etc.

Socket.IO is used only for real-time collaboration.

3. Authentication Flow

User registers or logs in.

```
Frontend
  |
POST /auth/login
  |
Backend verifies credentials
  |
JWT Generated
  |
Frontend stores token
  |
Authorization: Bearer <token>
```

Every protected API verifies the JWT using middleware.

Socket.IO also authenticates using the same JWT during connection.

4. Board Permissions

Every board has

- Owner
- Members

Owner can

- Rename board
- Delete board
- Invite members
- Approve join requests

Members can

- Draw
- Chat
- Use AI
- Collaborate

All board APIs first verify that the current user is a board member.

5. Real-Time Collaboration

Users join a Socket.IO room.

```
Board
|
Socket Room
|
User A
User B
User C
```

Whenever a user performs an action,

```
Draw
Text
Sticky
Arrow
Undo
Redo
Chat
Cursor
Typing
```

Frontend emits a socket event.

Backend

1. validates
2. stores action
3. broadcasts to everyone in the room

All connected users immediately see the update.

6. Canvas Architecture

The canvas uses **Event Sourcing**.

Instead of storing

```
Current Canvas
```

the backend stores

Draw

Rectangle

Text

Arrow

Undo

Redo

Sticky

Every interaction is stored as an immutable action.

Example

```
{
  "type": "draw",
  "payload": {
    "points": [...],
    "color": "#222"
  }
}
```

Nothing modifies previous actions.

New actions are always appended.

7. Why Event Sourcing?

Benefits

- Complete history
 - Undo/Redo
 - Easy replay
 - New users can reconstruct canvas
 - No need to continuously save snapshots
-

8. Action Types

Current supported actions

- draw
- shape
- text
- sticky
- arrow
- undo
- redo

Each action stores only the minimum information required to recreate itself.

Example

Rectangle

```
{
  "x": 100,
  "y": 120,
  "width": 200,
  "height": 80
}
```

Freehand drawing

```
{
  "points": [
    100,
    120,
    102,
    121,
    105,
    126
  ]
}
```

The points array is directly consumed by React Konva's `<Line>` component.

9. Board Reconstruction

When someone joins a board,

Backend returns

```
GET /board/:id/replay
```

which returns

```
Action1  
  
Action2  
  
Action3  
  
...  
  
ActionN
```

Frontend runs

```
buildBoardState(actions)
```

This reconstructs

```
lines[]  
  
rectangles[]  
  
texts[]  
  
stickies[]  
  
arrows[]
```

Undo actions remove previous objects.

Redo actions add them back.

Finally React Konva renders these arrays.

10. React Konva Rendering

Canvas state becomes

```
lines[]  
rectangles[]  
texts[]  
arrows[]  
stickies[]
```

Rendering is simply

```
lines.map()      -> <Line>  
  
rectangles.map() -> <Rect>  
  
texts.map()      -> <Text>  
  
arrows.map()     -> <Arrow>  
  
stickies.map()   -> Sticky Component
```

Konva converts these objects into pixels on the HTML5 Canvas.

11. Concurrency

Current approach is **real-time synchronization**, not CRDT.

Most actions are additive.

Example

```
User A creates Rectangle  
  
User B creates Circle
```

Both are stored independently.

No conflict occurs.

If two users modify the same object simultaneously,

the current implementation effectively follows **Last Writer Wins** based on server-side action ordering.

Future improvements

- CRDT
- Operational Transformation
- Object versioning
- Object locking

12. AI Architecture

AI runs as an independent FastAPI service.

```
graph TD
    Frontend --> Express
    Express --- Backend
    Express --> FastAPI
    FastAPI --- AI_Service[AI Service]
    FastAPI --> Gemini
```

Keeping AI separate makes the backend lightweight and allows the AI service to evolve independently.

13. AI Diagram Generation

User enters

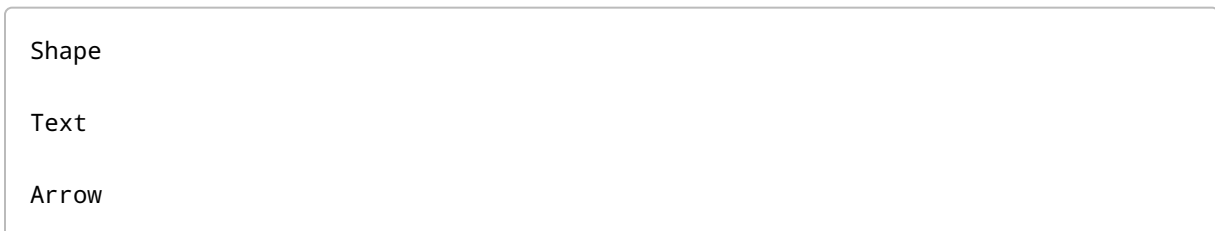
```
"Generate AWS architecture"
```

Flow

```
graph TD
    Frontend --> Express
```



Gemini returns structured actions such as



Backend enriches them with ids before saving.

Generated diagrams become fully editable because they are stored exactly like normal user actions.

14. AI Board Summarization

The backend first reconstructs the **current board state** (respecting undo/redo) instead of sending the raw action history.

It converts the board into a compact, readable description such as

Rectangle at (120,80)

Arrow connecting A to B

Sticky Note

Text

This description is sent to Gemini.

Gemini returns

- Main topic
- Important concepts
- Relationships
- Workflow
- Missing information

Frontend displays the markdown summary in a dialog.

15. Why Not Send Raw JSON to AI?

Advantages

- Smaller prompt
 - Lower token usage
 - Easier for LLM to understand
 - Better summaries
 - No unnecessary implementation details
-

16. Database Design

Collections

- Users
- Boards
- Actions
- JoinRequests

Relationships

Board

└ Owner

└ Members

└ Actions

Actions are separated from boards because they continuously grow.

17. Backend Architecture

The backend follows a service-oriented architecture.

Routes

↓

Middleware

↓

Services

↓

Models

↓

MongoDB

Responsibilities

Routes

- Request validation
- Response

Services

- Business logic

Models

- Database

Middleware

- Authentication
- Authorization

This separation keeps the code modular and maintainable.

18. Current Features

- JWT Authentication
 - Guest Mode
 - Board Ownership
 - Member Invitations
 - Join Request Approval
 - Real-Time Drawing
 - Freehand Drawing
 - Shapes
 - Text
 - Sticky Notes
 - Arrows
 - Chat
 - Live Cursor
 - Presence Indicators
 - Typing Indicators
 - Undo / Redo
 - Event Replay
 - AI Diagram Generation
 - AI Board Summarization
 - Export Canvas
-

19. Possible Future Improvements

- CRDT-based conflict resolution
- LangGraph AI agents
- AI-powered board editing
- AI Q&A over board
- Canvas snapshots
- Event batching
- Infinite canvas
- Version history

- Object resizing and rotation
 - Offline collaboration
 - Background cleanup jobs for guest sessions
-

20. One-Line Project Summary

DrawMeet is a real-time collaborative whiteboard built on an event-sourced architecture where every canvas operation is stored as an immutable action in MongoDB, synchronized via Socket.IO, reconstructed through replay, rendered with React Konva, and enhanced with an independent FastAPI + Gemini AI service for diagram generation and intelligent board summarization.